SIMATS ENGINEERING
Department of ComputerScience and Engineering ASSESSMENT TOOL2 - CONCEPT MAPPING

Course Code:	CSA1205	**Course Title:**	Computer Architecture for Multicore Systems
CO Assessed:	**CO1**	**Assessment:**	**ASSESSMENT TOOL 2-** CONCEPT MAPPING
Weightage:	**35%**	**Total Marks:**	**25**
CO1 Statement:	Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)		
Student Name:		**Reg.No.:**	
Department:		**Date:**	

Instruction to Students

1. Answer two questions as per the Instruction.
2. Each question carries 12.5 mark.
3. Only the appropriate Tools can be used
4. Time allotted for the exam is 60 min

ANNEXURE A

1. Construct a detailed concept map showing the relationship between CPU (ALU, Control Unit, Registers), memory (cache, RAM), and I/O devices. Extend the map to include single-bus, multi-bus, and hierarchical bus structures, and clearly illustrate how data and control signals flow among these components during execution. Indicate possible points of delay or bottlenecks and how different bus structures help in improving performance.
2. Develop a comprehensive concept map linking the instruction execution cycle (fetch, decode, execute) with CPU performance parameters such as CPI, clock cycles, and execution time. Show how each stage contributes to overall execution time, and include connections to factors like memory access, instruction complexity, and pipeline stalls. Also, represent how benchmarking is used to evaluate performance.
3. Create a concept map comparing 8085 and 8086 architectures, including their instruction sets, register organization, addressing modes, and memory segmentation (in 8086). Clearly show how these architectural features influence program execution, multitasking capability, and efficiency in real-time applications.

4. Draw a detailed concept map that connects various addressing modes (immediate, direct, register, indirect, indexed) with instruction types (data transfer, arithmetic, control instructions). Illustrate how each addressing mode affects memory access time, instruction size, and execution speed, and show real-world scenarios where specific addressing modes are preferred.
5. Prepare an elaborate concept map integrating number systems (binary and hexadecimal) with instruction representation, memory storage, and debugging processes. Show how data is converted between formats, how instructions are stored and interpreted by the CPU, and why hexadecimal representation is commonly used in low-level programming and system design.

Code of Conduct:

I Anushya devi V (192424013) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Anushya devi V

Signature of the student:

MARKS ALLOCATION

Criteria	Concept Identification (25%)	Linkages (25%)	Organization (20%)	Integration of Literature (20%)	Clarity (10%)
Q1					
Q2					

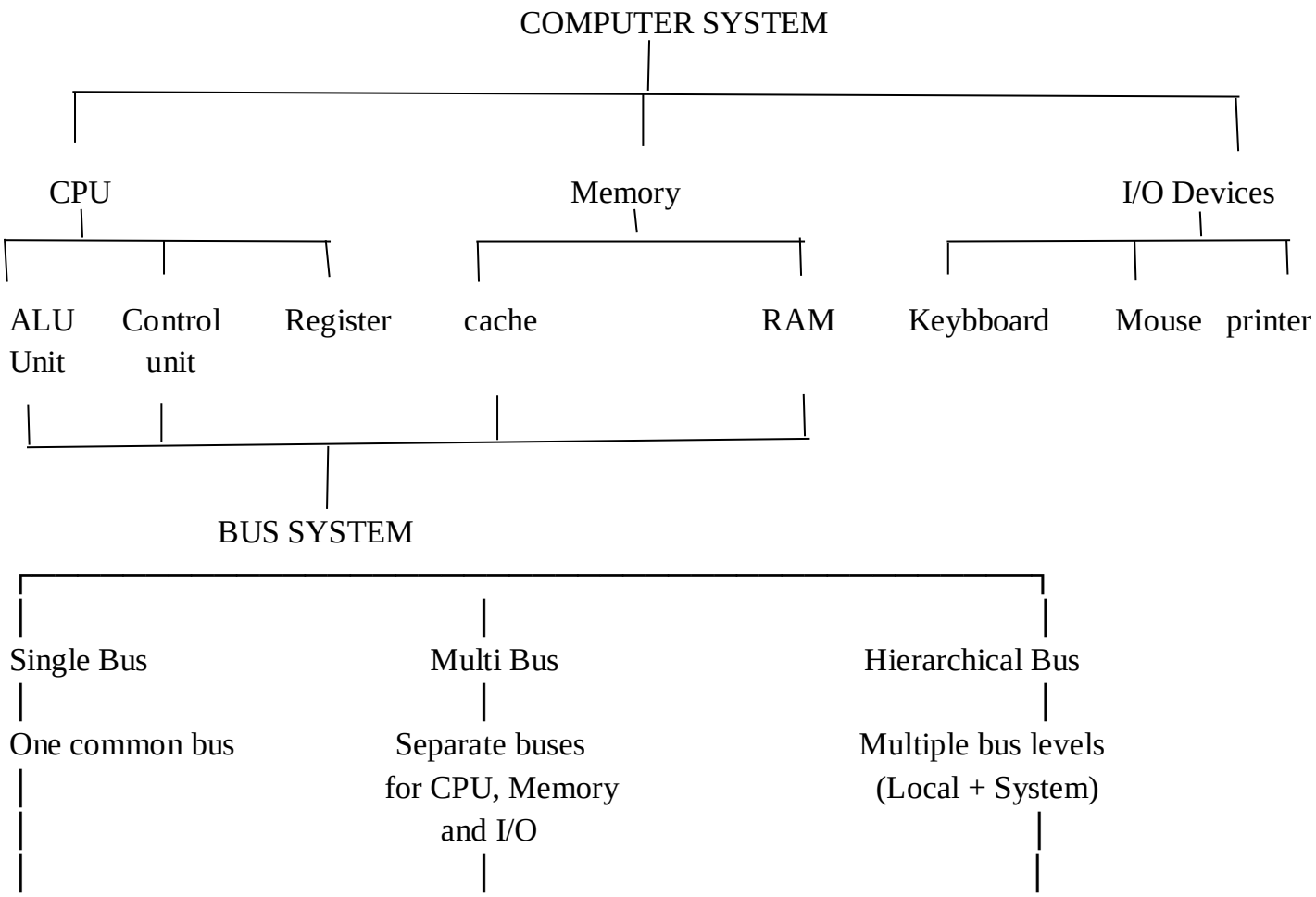
RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established

Organization	20%	Concepts are well structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

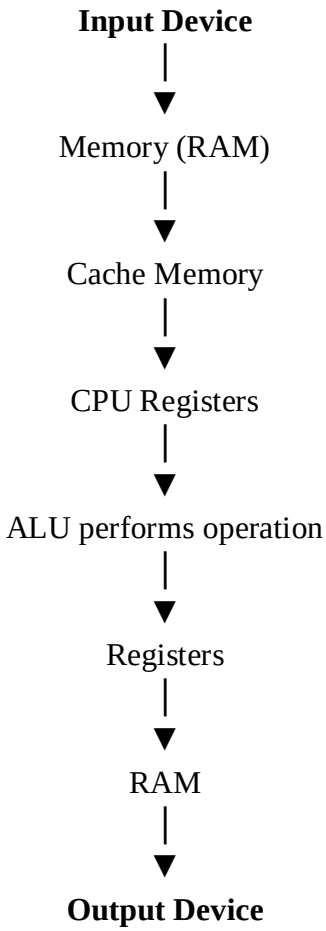
ANSWER:

1. Concept Map: CPU, Memory, I/O Devices and Bus Structures



Slow communication	Faster transfer	Highest performance
Bus contention	Parallel transfer	Reduced congestion
Higher delay	Less waiting	Better scalability

Data Flow During Execution



Control Signals

Control Unit

```
│
├── Memory Read
├── Memory Write
├── I/O Read
└── I/O Write
```

Possible Bottlenecks

- Slow RAM access
- Bus congestion
- Cache miss
- Slow I/O devices

Performance Improvement

Single Bus

↓

More waiting

Multi Bus

↓

Parallel communication

Hierarchical Bus

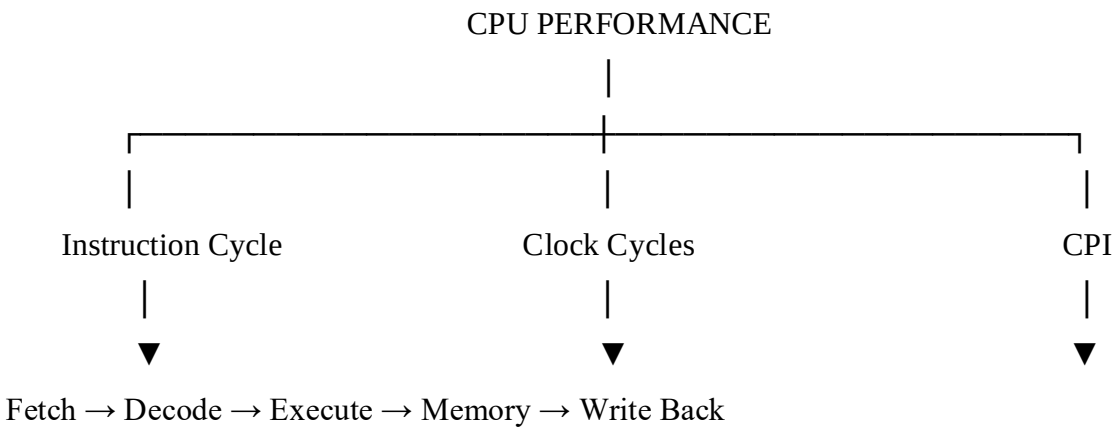
↓

Reduced traffic

Higher speed

Better system performance

2. Concept Map: Instruction Execution Cycle and CPU Performance

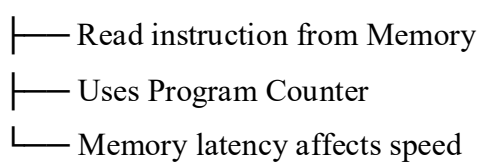


Execution Time

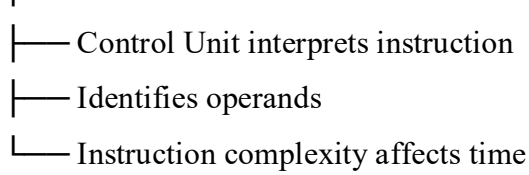
Execution Time =

Instruction Count × CPI × Clock Cycle Time

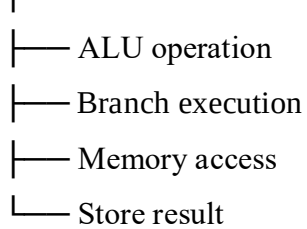
Fetch



Decode



Execute



Factors Affecting Performance

Memory Access

Instruction Complexity
│
└── Complex instruction → Higher CPI

Pipeline Stalls
│
├── Data Hazard
├── Control Hazard
└── Structural Hazard

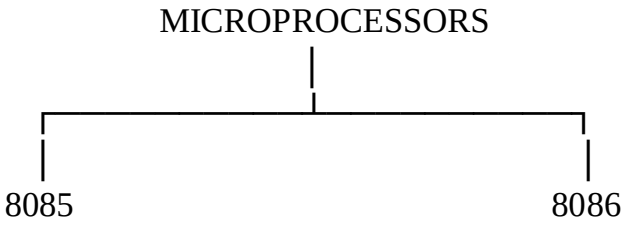
Clock Frequency
│
└── Higher Frequency → Lower Execution Time

Benchmarking

Benchmarks
│
├── SPEC CPU
├── Dhrystone
├── Whetstone
└── Application Benchmarks

Benchmark Results
│
├── Execution Time
├── Throughput
├── CPI
└── CPU Efficiency

3. Concept Map: Comparison of 8085 and 8086 Architecture



8085

- 8-bit Processor
- 16-bit Address Bus
- 64 KB Memory
- Accumulator-based
- No Memory Segmentation
- Simple Instruction Set
- Limited Multitasking

8086

- 16-bit Processor
- 20-bit Address Bus
- 1 MB Memory
- Memory Segmentation
  - Code Segment
  - Data Segment
  - Stack Segment
  - Extra Segment
- BIU (Bus Interface Unit)
- EU (Execution Unit)
- Instruction Queue
- Faster Execution

Registers

8085

- A
- B
- C
- D
- E
- H
- L

8086

- AX
- BX
- CX
- DX
- SI
- DI
- SP
- BP
- IP
- Flags

Addressing Modes

8085

- Immediate
- Register
- Direct
- Indirect

8086

- Immediate
- Register
- Direct

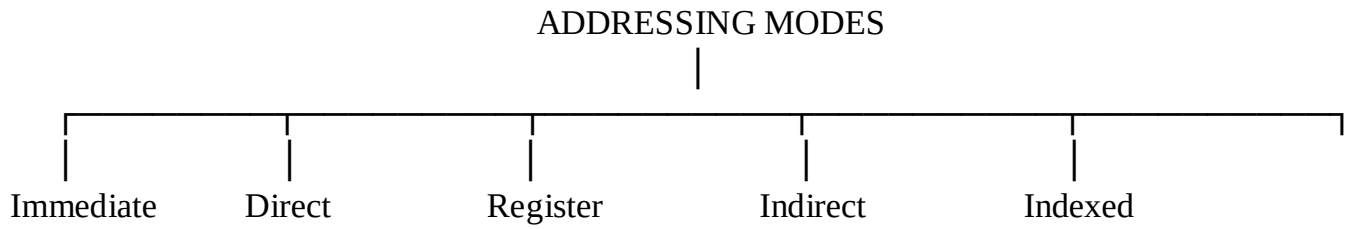
├── Indirect
├── Indexed
└── Based

Effects

8085
│
├── Slower execution
├── Limited applications
└── Simple embedded systems

8086
│
├── Faster execution
├── Better multitasking
├── Larger memory
└── Real-time applications

4. Concept Map: Addressing Modes and Instruction Types



Instruction Types

Data Transfer
│
├── MOV
├── LOAD
└── STORE

Arithmetic
│
├── ADD
├── SUB
├── MUL
└── DIV

Control
│
├── JMP
├── CALL
├── RET
└── LOOP

Immediate
│
├── Operand inside instruction
├── Fast execution
├── Small memory access
└── Example:
  MOV A,25H

Direct
│
├── Memory address given
├── Extra memory access
└── Example:
  MOV A,[2000H]

Register
│
├── Operand in register
├── Fastest
└── Example:
  ADD AX,BX

Indirect
│
├── Address stored in register
├── Flexible
└── Example:
MOV A,[BX]

Indexed
│
├── Base + Index
├── Array processing
└── Example:
MOV AX,[SI+100]

Performance

Immediate
│
├── Small instruction
├── Fast execution
└── Less memory access

Register
│
├── Fastest
└── No RAM access

Direct
│
├── Moderate speed
└── Memory required

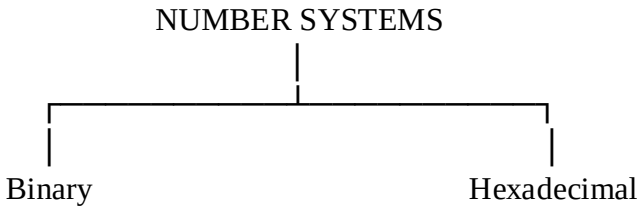
Indirect
│
├── More flexible
└── Additional memory access

Indexed
│
├── Efficient for arrays
└── Medium execution time

Real-world Applications

- Immediate → Constant values
- Register → Mathematical calculations
- Direct → Fixed memory locations
- Indirect → Linked Lists
- Indexed → Arrays and Tables

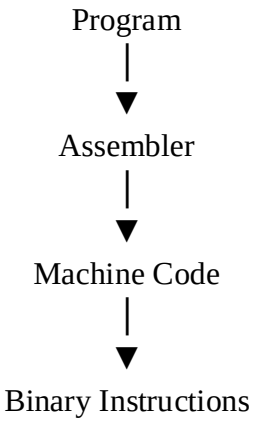
5. Concept Map: Number Systems, Memory Storage and Debugging



Binary
│
├── 0 and 1
├── Used by CPU
└── Digital circuits

Hexadecimal
│
├── 0–9
├── A–F
├── Compact form
└── Easier to read

Instruction Representation



Memory Storage

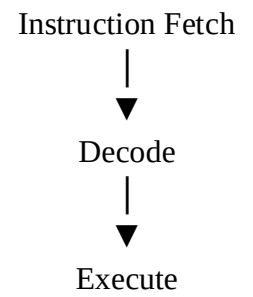
Memory
- Bytes
- Words
- Binary Data
- Hex Addresses

Example

Address → 1000H

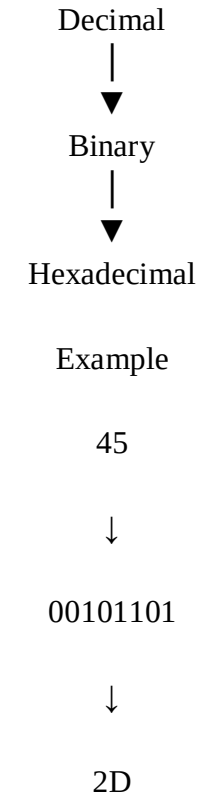
Data → 3FH

CPU Interpretation

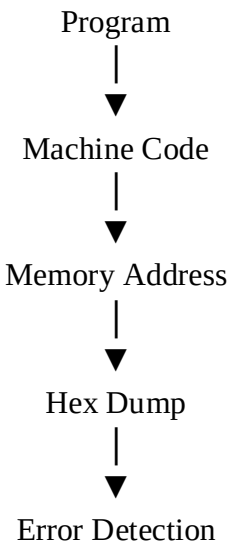


CPU understands only Binary

Conversion



Debugging



Why Hexadecimal?